PEcb09. Отчёт и инструкция запуска

Обязательная часть

Готовый ответ из **7 предложений** находится в файле `Обязательная_часть.txt`. Он составлен по коду обоих приложенных архивов, а не только по описанию урока.

Дополнительная часть

Проект запущен и проверен локально: настоящий FastAPI-сайт, HTTP-запросы через исходный `client.py`, исходные `worker.py`, `processor.py`, `state.py` и локальная база SQLite. Отзывы добавлены через POST API сайта; ответы опубликованы как дочерние комментарии. VPS не использовался.

API-ключ намеренно пустой: реальный вызов LLM не выполнялся. Использован предусмотренный исходным `processor.py` локальный fallback, то есть **шаблонный mock генерации ответа**, а не нейросеть. Тональность определена исходным классификатором по ключевым словам. Это проверяет взаимодействие компонентов, но не качество LLM и не доступность внешнего API.

Проверены три отзыва: позитивный («Отлично, спасибо! Товар понравился, доставка быстро.»), негативный («Ужас, товар не работает, я недоволен.») и нейтральный («Заказ получен во вторник. Упаковка серая.»). Для позитивного шаблон благодарит за добрые слова, для негативного выражает сожаление и предлагает обратиться в поддержку, для нейтрального благодарит и предлагает поделиться деталями. Последний ответ универсальный и не учитывает конкретные детали отзыва - это ограничение mock.

Отдельные результаты: `Таблица_тестирования.csv`, `test_results.json`, снимок API `reviews_snapshot.json`, журналы `test_run.log` и `server.log`. Проверяются точное совпадение ожидаемой тональности, статус processed, один непустой ответ на каждый отзыв, статус ответа processed и отметка в state file. Дополнительно проверяются повторный опрос после перечитывания состояния и защита от дубля при искусственном возврате уже обработанного отзыва в new. После проверки статус восстановлен.

Telegram отключён: реальные уведомления не отправлялись и их доставка не проверялась. В исходнике уведомления предусмотрены для всех новых пользовательских отзывов, не только негативных. Важно: `mark_processed` добавляет ID также в список notified даже при отключённом Telegram; наличие ID в этом списке само по себе не доказывает доставку уведомления.

Что изменено

В `app/config.py` добавлен выбор локального URL базы; в `app/main.py` PostgreSQL-специфичные ALTER TABLE выполняются только для PostgreSQL. Новая SQLite-база получает все поля через create_all. Функциональные модули обработчика сохранены без изменений. В `.env` нет реальных секретов: ключи пустые, worker-токен - публичная учебная строка только для локальной проверки. Исходники архивов сохранены в `source/`.

Запуск выполнен непосредственно через Python, **не через Docker Compose**. Контейнерный запуск из задания не проверен; это локальная адаптация для проверки пайплайна без PostgreSQL и внешних сервисов. Сервер после теста остановлен, база и результаты сохранены.

Повторение проверки

Из этой папки в PowerShell:

```
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements-local.txt
.\.venv\Scripts\python.exe test_pipeline.py
```

Каждый повторный запуск добавляет ещё три тестовых отзыва и проверяет их; прежние processed записи не обрабатываются повторно. Тест принудительно очищает переменные ключей в своём процессе, поэтому не использует даже случайно унаследованные секреты. Установка зависимостей требует сети, сама проверка использует только 127.0.0.1. Порт 18009 должен быть свободен.

Чтобы открыть сайт вручную:

```
.\.venv\Scripts\python.exe -m uvicorn app.main:app --host 127.0.0.1 --port 18009
```

Адрес: http://127.0.0.1:18009. В другом терминале из этой же папки можно запустить `.\.venv\Scripts\python.exe worker/worker.py`; настройки берутся из `.env`.

Ограничения исходного решения

Маркеры не понимают отрицание и сарказм; три успешных примера не доказывают качество на произвольных отзывах. POST ответа, PATCH статусов и запись JSON не являются одной транзакцией, поэтому авария между ними может привести к дублю. State file не рассчитан на несколько параллельных воркеров. Эти свойства не скрываются результатами успешного обычного повторного опроса.